Two Dimensional Array

13 July 2026 16:45

A two dimensional (2D) array is an array whose elements are themselves arrays. It is commonly used to represent data in the form of rows and columns, just like a table, spreadsheet, chess board, sitting arrangement or matrix.

For example:

Roll no	Name	Marks
101	Amit	89
102	Priya	95
103	Rahul	78

This tabular data can be represented using a 2D array.

Declaration of a 2D Array
SyntaxL

```
dataType[][] arrayName;
      Or
dataType arrayName[][];
```

Example:

```
int[][] marks;
```

This statement only declares the array reference, no memory is allocated.

Creating a 2D Array
Memory is allocated using the **new** keyword.

Syntax:

```
Arrayname = new dataType[rows][columns];
```

Example:

```
marks = new dataType[3][4];
      Or
int[][] marks = new dataType[3][4];
```

This creates a 2D array with:

- **3 rows**
- **4 columns**
- **Total elements = 3 * 4 = 12**

Memory representation:

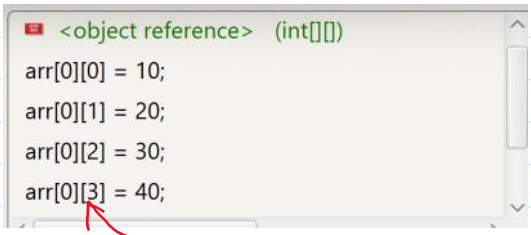
Row Column

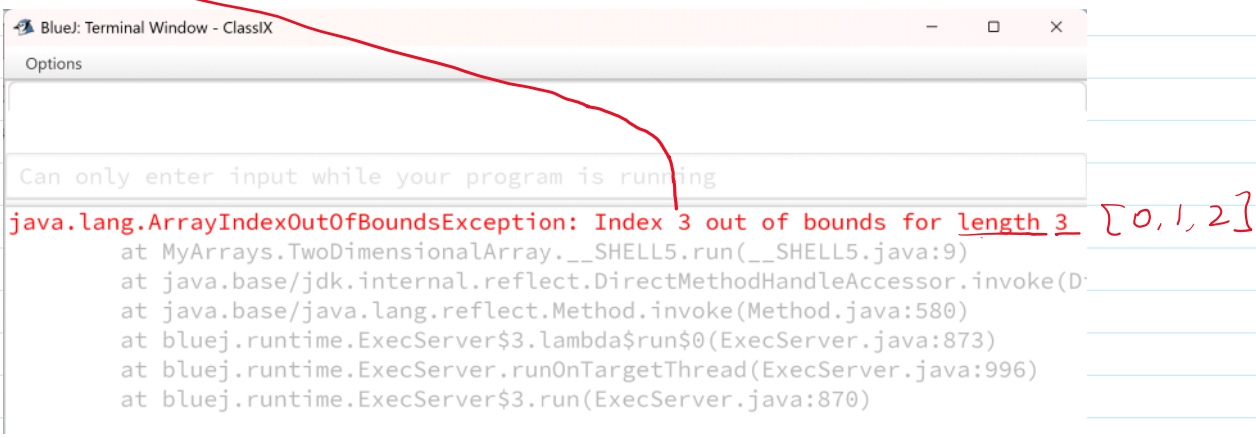
0,0	**0,1**	**0,2**	**0,3**
1,0	**1,1**	**1,2**	**1,3**
2,0	**2,1**	**2,2**	**2,3**

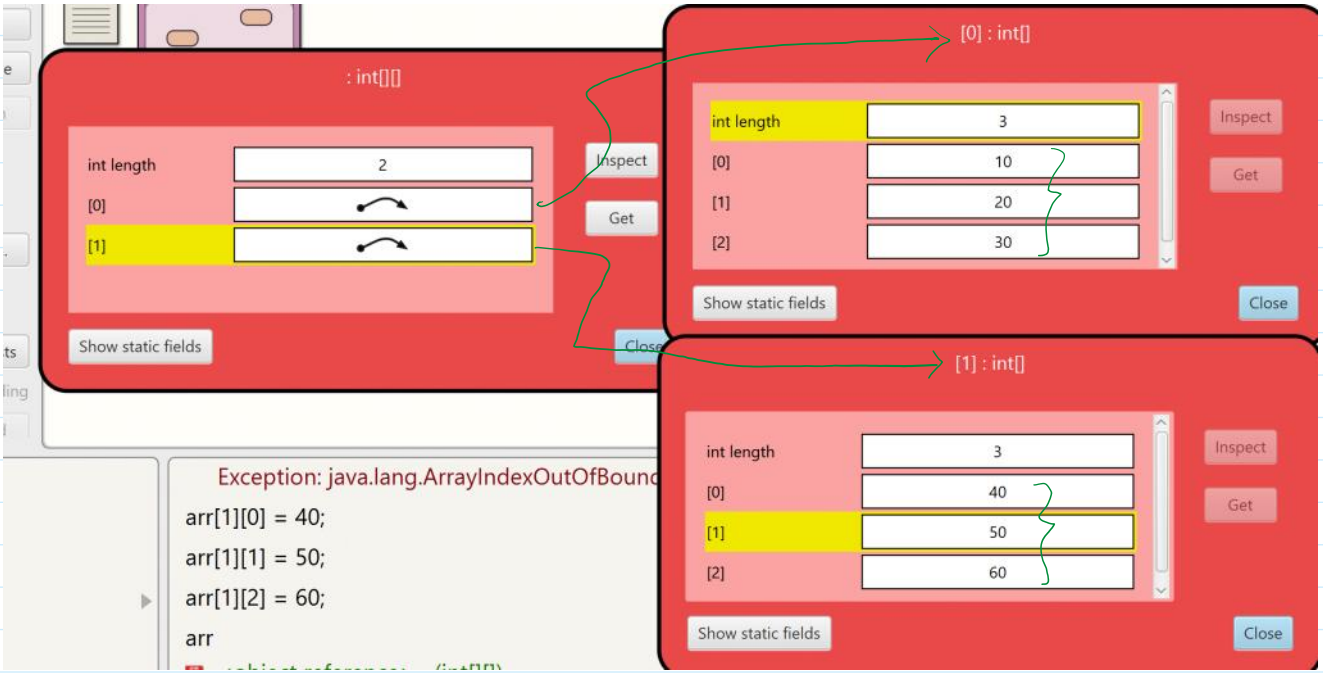
0
1
2

1,0	1,1	1,2	1,3
2,0	2,1	2,2	2,3

Initializing a 2D array
Method 1: Using Assignment





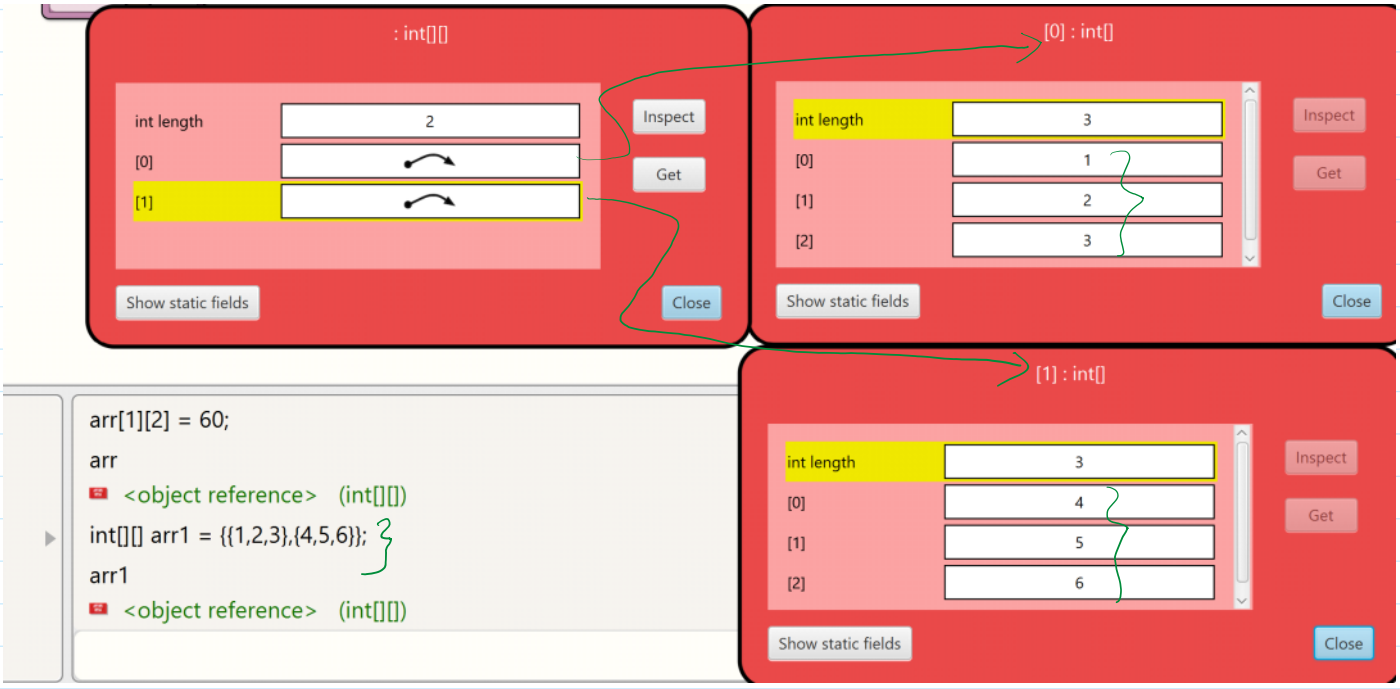


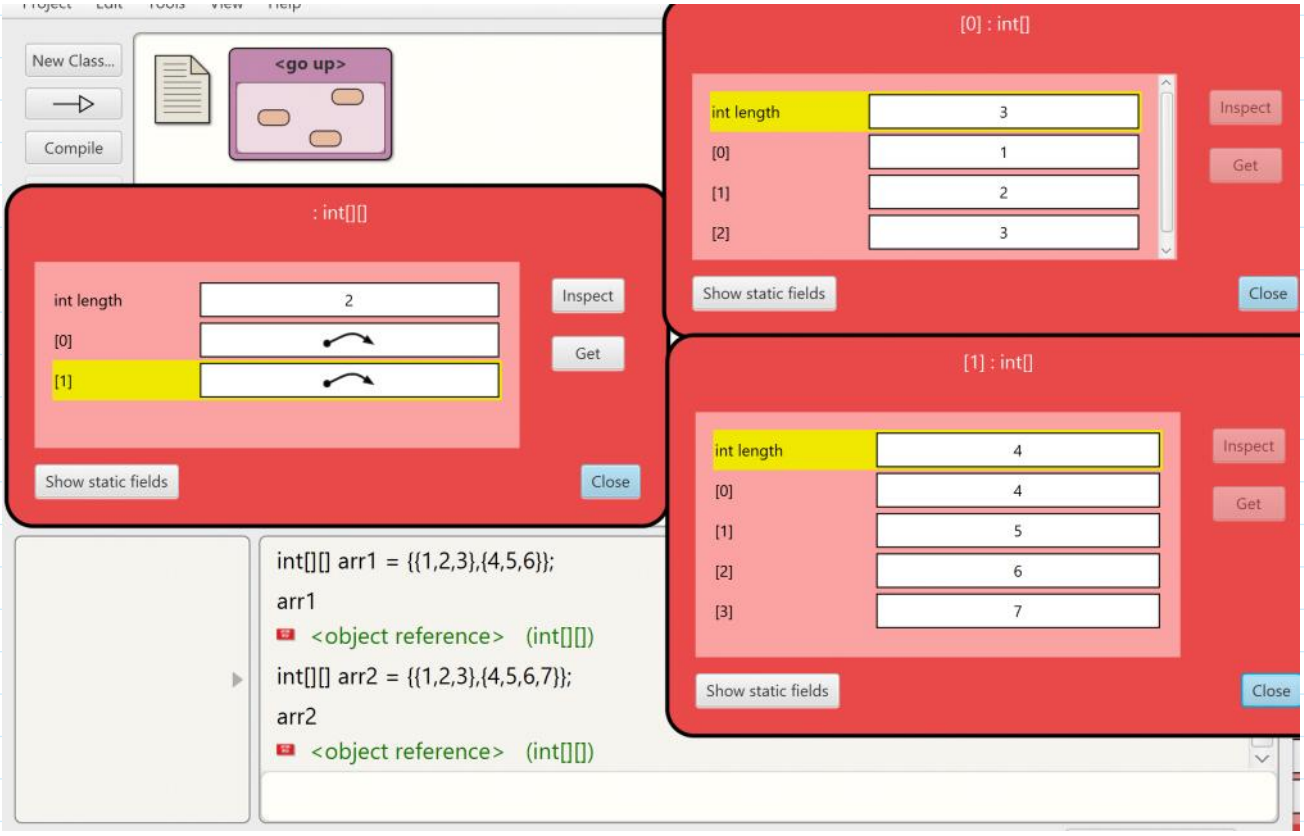

```
int[][] arr = new int[2][3];
arr
<object reference>   (int[][])
arr[0][0] = 10;
```

```
arr[0][1] = 20;
arr[0][2] = 30;
arr[0][3] = 40;
Exception: java.lang.ArrayIndexOutOfBoundsException (Index 3 out of bounds for
length 3)
arr[1][0] = 40;
arr[1][1] = 50;
arr[1][2] = 60;
arr
<object reference>   (int[][])
```

Method 2: Using Array Initializer

```
int[][] arr = {
   {10, 20, 30},
   {40, 50, 60}
};
```





Accessing Elements

Syntax:

```
arrayName[row][column];
```

Example:

```
System.out.print("\n"+arr2[0][2]);
```

Output:

```
3
```

Reading Values from the User

```
package MyArrays.TwoDimensionalArray;
import java.util.*;


/**
 * Write a description of class TwoDArrayUtil here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class TwoDArrayUtil
{
    private static Scanner sc = new Scanner(System.in);
    /**
     * This function takes data input in each element from the user.
     * @param arr int array in which data will inserted by the user
```

```
     * @param msg Any message or text displayed on the terminal.
     */
    public static void inputInt2DArray(int[][] arr, String msg){
        System.out.print("\n" + msg + "\n");
        int i, j;
        for(i = 0; i < arr.length; i++){
            for(j = 0; j < arr[i].length; j++){
                System.out.print("Enter integer value in ["+i+"]["+j+"]: ");
                arr[i][j] = sc.nextInt();
            }
        }
    }

    /**
     * This functions display on terminal int 2D array in matrix format
     * @param arr Two dimensional int array.
     * @param msg Any String message print with Array.
     */
    public static void printInt2DArray(int[][] arr, String msg){
        System.out.print("\n" + msg + "\n");
        int i, j;
        for(i = 0; i < arr.length; i++){
            for(j = 0; j < arr[i].length; j++){
                System.out.print("\t" + arr[i][j]);
            }
            System.out.print("\n");
        }
    }
}

package MyArrays.TwoDimensionalArray;
import java.util.*;


/**
 * Write a description of class Test2DArray here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Test2DArray
{
    public static void main(String[] args){
        int rows, columns;
        Scanner sc = new Scanner(System.in);
        System.out.print("\fEnter number of rows: ");
        rows = sc.nextInt();
        System.out.print("Enter number of columns: ");
        columns = sc.nextInt();
        int[][] A = new int[rows][columns];
        TwoDArrayUtil.inputInt2DArray(A, "Enter values in A[][]:");
```

number of rows in 2-D array

number of columns in 2-D array

Traversing of Array

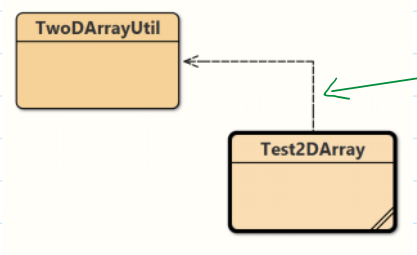
```
        TwoDArrayUtil.printInt2DArray(A, "Array A[][]: ");
    }
}
```

Output:

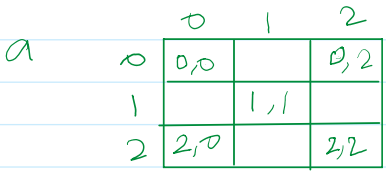
```
BlueJ: Terminal Window - ClassIX
Options

Enter values in A[][]:
Enter integer value in [0][0]: 10
Enter integer value in [0][1]: 12
Enter integer value in [0][2]: 14
Enter integer value in [1][0]: 16
Enter integer value in [1][1]: 18
Enter integer value in [1][2]: 20

Array A[][]:
        10      12      14
        16      18      20
```



Test2DArray class containing the reference of TwoDArrayUtil class. This concept is called Containership in OOP



a[i][i] ← left diagonal

a[i][a.length - 1 - i] ← right diagonal

Array Questions in Java

1. **Matrix Addition** Write a program to input two matrices of size N x M. Perform element-wise addition and print the resulting matrix in tabular form. Extend the program to also compute the difference of the matrices.

2. **Matrix Multiplication** Accept two matrices A[m x n] and B[n x p]. Multiply them to produce matrix C[m x p]. Validate dimensions before multiplication. Print the result neatly.

3. **Transpose of Matrix** Input a matrix and print its transpose. Extend the program to check if the matrix is symmetric (A[i][j] == A[j][i]).

4. **Diagonal Operations** For a square matrix, compute:
    - Sum of main diagonal
    - Sum of secondary diagonal
    - Difference between them

5. **Spiral Traversal** Input a matrix and print its elements in spiral order (clockwise). Extend to anti-clockwise spiral traversal.

6. **Snake Pattern Traversal** Print elements row by row, alternating left-to-right and right-to-left. Example:
   Code
   1 2 3
   6 5 4
   7 8 9
7. **Row with Maximum Sum** Find the row with the maximum sum. Print the row index, sum, and elements of that row.

8. **Column with Maximum Sum** Find the column with the maximum sum. Print the column index, sum, and elements of that column.

9. **Search Element** Input a matrix and search for a given element. Print its position (row, column). If not found, display "Element not present".

10. **Rotate Matrix 90°** Rotate a square matrix clockwise by 90°. Extend the program to rotate by 180° and 270°.

11. **Boundary Traversal** Print only the boundary elements of a matrix. Example:
    Code
    1 2 3
    4 5
    6 7 8
12. **Magic Square Check** Input a square matrix. Check if it is a magic square (sum of rows, columns, and diagonals are equal).

13. **Pascal's Triangle** Generate Pascal's Triangle using a 2D array. Print it in triangular format.

14. **Sudoku Validator** Input a 9x9 Sudoku grid. Validate if:
    - Each row contains digits 1–9 without repetition.
    - Each column contains digits 1–9 without repetition.
    - Each 3x3 subgrid contains digits 1–9 without repetition.

15. **Image Pixel Manipulation** Represent a grayscale image as a 2D array (values 0–255). Implement:
    - Brightness increase/decrease.
    - Horizontal flip.
    - Vertical flip.
    - Inversion (255 – value).